Football Tickets API: ТЗ и пошаговая инструкция

Проект №1 для GitHub-портфолио. Стек: C#, ASP.NET Core, EF Core, PostgreSQL, Docker.

Часть 1. Техническое задание

1.1 Цель

Бэкенд онлайн-сервиса продажи билетов на футбольные матчи. Пользователи смотрят матчи и схему мест, покупают билеты и отменяют заказы. Администратор управляет командами, стадионами и матчами.

Цель проекта для вас: показать на GitHub слоистую архитектуру, работу с БД и миграциями, авторизацию, тесты, Docker и CI. Доменная область та же, что в ваших лабах по Enterprise Architect, так что диаграммы можно положить в папку docs и сослаться на них в README.

1.2 Стек

- .NET 10 SDK (подойдет и 8 или 9, команды те же)
- ASP.NET Core Web API на контроллерах
- EF Core + PostgreSQL (провайдер Npgsql), миграции
- JWT-авторизация, роли User и Admin
- FluentValidation для валидации входных данных
- Serilog для логов
- xUnit + Moq для unit-тестов, WebApplicationFactory для интеграционных
- Docker и docker-compose, GitHub Actions
- Scalar для документации API (в .NET 9+ шаблон уже отдает OpenAPI-описание, но без интерфейса)

MediatR и AutoMapper не берем: в новых версиях они перешли на коммерческие лицензии, а для проекта такого размера хватит обычных сервисов и ручного маппинга. Это еще и проще объяснить на собеседовании.

1.3 Сущности и таблицы

Сущность	Поля	Примечание
User	Id (Guid), Email, PasswordHash, FullName, Role, CreatedAt	Email уникален
Team	Id, Name, City	Name уникален
Stadium	Id, Name, City	
Sector	Id, StadiumId, Name, PriceMultiplier	Множитель цены сектора, например 1.0, 1.5, 0.7
Seat	Id, SectorId, Row, Number	Уникальны (SectorId, Row, Number)
Match	Id, HomeTeamId, AwayTeamId, StadiumId, StartsAt (UTC), BasePrice, Status	Status: Scheduled, Cancelled, Finished
Order	Id, UserId, CreatedAt, Status, TotalAmount	Status: Paid, Cancelled
Ticket	Id, OrderId, MatchId, SeatId, Price, IsCancelled	Частичный уникальный индекс (MatchId, SeatId) только для IsCancelled = false

Связи: Stadium 1-N Sector, Sector 1-N Seat, Team 1-N Match (как домашняя и как гостевая), Stadium 1-N Match, User 1-N Order, Order 1-N Ticket, Match 1-N Ticket, Seat 1-N Ticket.

1.4 Бизнес-правила

1. Один заказ относится к одному матчу и содержит от 1 до 6 мест.
2. Нельзя купить билет на матч, который уже начался или отменен.
3. Одно место нельзя продать дважды на один матч. Гарантия дает уникальный индекс в БД, а не только проверка в коде (иначе при двух одновременных запросах место уйдет обоим).
4. Цена билета = Match.BasePrice * Sector.PriceMultiplier. Цена фиксируется в билете в момент покупки.
5. Заказ может отменить только его владелец и не позднее чем за 24 часа до начала матча. При отмене билеты помечаются IsCancelled, места освобождаются.
6. Матч нельзя удалить, если на него есть активные билеты.

7. Оплата эмулируется: заказ сразу создается со статусом Paid.
8. Регистрация: уникальный email, пароль не короче 8 символов, пароль хранится только в виде хэша.
9. Даты хранятся и передаются в UTC.

1.5 Эндпоинты

Метод	URL	Доступ	Описание
POST	/api/auth/register	все	Регистрация, возвращает JWT
POST	/api/auth/login	все	Вход, возвращает JWT
GET	/api/matches	все	Список матчей, фильтры from, to, teamId, пагинация page и pageSize
GET	/api/matches/{id}	все	Карточка матча
GET	/api/matches/{id}/seats	все	Места по секторам со статусом свободно или занято и ценой
POST	/api/matches	Admin	Создать матч
PUT	/api/matches/{id}	Admin	Изменить матч
DELETE	/api/matches/{id}	Admin	Удалить матч (правило 6)
GET, POST, PUT, DELETE	/api/teams	GET все, остальное Admin	Команды
GET	/api/stadiums	все	Стадионы с секторами
POST	/api/orders	User	Купить билеты: matchId и список seatIds
GET	/api/orders/my	User	Мои заказы с билетами
GET	/api/orders/{id}	User	Заказ (только свой)
POST	/api/orders/{id}/cancel	User	Отменить заказ (правило 5)

1.6 Нефункциональные требования

- Все методы асинхронные, в контроллерах нет бизнес-логики.
- Единый формат ошибок ProblemDetails и коды ответов 400, 401, 403, 404, 409.
- Секреты (строка подключения, ключ JWT) не лежат в репозитории.
- Схема БД создается миграциями, тестовые данные заливаются сидером.
- Проект запускается одной командой docker compose up.
- В README есть описание, схема архитектуры, ER-диаграмма, инструкция запуска и раздел "Что бы улучшил".

Часть 2. Архитектура

2.1 Структура решения

```
football-tickets-api/
├─ src/
|  ├─ FootballTickets.Domain/
|  ├─ FootballTickets.Application/
|  ├─ FootballTickets.Infrastructure/
|  └─ FootballTickets.Api/
├─ tests/
|  ├─ FootballTickets.Application.Tests/
|  └─ FootballTickets.Api.IntegrationTests/
├─ docs/
├─ docker-compose.yml
├─ README.md
└─ FootballTickets.sln (в .NET 10 может быть .slnx, это нормально)
```

2.2 Кто за что отвечает

- **Domain**: сущности и перечисления. Ни от чего не зависит, внутри чистые классы.
- **Application**: интерфейсы и сервисы с бизнес-логикой (MatchService, OrderService, AuthService), DTO, валидаторы, свои исключения. Зависит только от Domain. Для работы с БД определяем интерфейс IAppDbContext (DbSet-ы и SaveChangesAsync), без репозиториев: EF Core и так реализует паттерн Repository и Unit of Work, лишний слой только раздует код.
- **Infrastructure**: реализация доступа к данным (AppDbContext, конфигурации сущностей, миграции, сидер), генерация JWT, хэширование паролей. Зависит от Application.

- **Api**: контроллеры, настройка DI, авторизация, обработка ошибок, Swagger/Scalar. Единственный проект, который знает обо всех остальных (нужно для регистрации сервисов).

Правило зависимостей: стрелки только внутрь (Api -> Infrastructure -> Application -> Domain). Если Domain или Application ссылаются наружу, архитектура сломана.

Часть 3. Пошаговая инструкция

Шаг 0. Что нужно установить

1. .NET SDK. Проверка: `dotnet --version`.
2. Visual Studio (или Rider, или VS Code с C# Dev Kit).
3. Git и аккаунт на GitHub.
4. Docker Desktop (для PostgreSQL и итогового запуска). Если Docker поставить не получается, можно поставить PostgreSQL обычным установщиком, дальше в инструкции разница будет только в строке подключения.
5. Клиент для БД: DBeaver или pgAdmin, чтобы смотреть таблицы глазами.
6. Инструмент миграций EF Core: `dotnet tool install --global dotnet-ef`.

Почему PostgreSQL, а не SQL Server: бесплатный, легкий в Docker, часто встречается в вакансиях. Если хотите SQL Server (как в лабах), замените пакет провайдера на Microsoft.EntityFrameworkCore.SqlServer и вызов UseNpgsql на UseSqlServer, остальной код не меняется.

Шаг 1. Репозиторий на GitHub

1. На GitHub создайте пустой репозиторий football-tickets-api (без README и .gitignore, их сделаем локально). Описание: "REST API for football ticket sales: ASP.NET Core, EF Core, PostgreSQL, Docker".
2. Локально откройте терминал (PowerShell) в папке, где будут проекты.

```
mkdir football-tickets-api
cd football-tickets-api
git init
git branch -M main
git remote add origin https://github.com/ВАШ_ЛОГИН/football-tickets-api.git
dotnet new gitignore
```

3. Создайте README.md с одной строкой-заголовком и сделайте первый коммит:

```
git add .
git commit -m "chore: init repository with gitignore and readme"
git push -u origin main
```

Правило на весь проект: один осмысленный шаг - один коммит, сообщения в стиле `feat:`, `fix:`, `test:`, `chore:`, `docs:`. По истории коммитов видно, как вы работали, и это один из главных аргументов для проверяющего.

Шаг 2. Создаем решение и проекты

```
dotnet new sln -n FootballTickets
mkdir src
mkdir tests
dotnet new classlib -n FootballTickets.Domain -o src/FootballTickets.Domain
dotnet new classlib -n FootballTickets.Application -o
src/FootballTickets.Application
dotnet new classlib -n FootballTickets.Infrastructure -o
src/FootballTickets.Infrastructure
dotnet new webapi -n FootballTickets.Api -o src/FootballTickets.Api --use-
controllers
dotnet new xunit -n FootballTickets.Application.Tests -o
tests/FootballTickets.Application.Tests
dotnet new xunit -n FootballTickets.Api.IntegrationTests -o
tests/FootballTickets.Api.IntegrationTests
```

Добавьте проекты в решение:

```
dotnet sln add src/FootballTickets.Domain/FootballTickets.Domain.csproj
dotnet sln add
src/FootballTickets.Application/FootballTickets.Application.csproj
dotnet sln add
src/FootballTickets.Infrastructure/FootballTickets.Infrastructure.csproj
dotnet sln add src/FootballTickets.Api/FootballTickets.Api.csproj
dotnet sln add
tests/FootballTickets.Application.Tests/FootballTickets.Application.Tests.csp
roj
dotnet sln add
tests/FootballTickets.Api.IntegrationTests/FootballTickets.Api.IntegrationTes
ts.csproj
```

Удалите заготовки шаблонов: `Class1.cs` во всех трех библиотеках, `UnitTest1.cs` в обоих тестовых проектах, а в Api файлы WeatherForecast.cs и WeatherForecastController.cs (если они есть).

Шаг 3. Ссылки между проектами

```
dotnet add src/FootballTickets.Application reference
src/FootballTickets.Domain
dotnet add src/FootballTickets.Infrastructure reference
src/FootballTickets.Application
dotnet add src/FootballTickets.Api reference src/FootballTickets.Application
dotnet add src/FootballTickets.Api reference
src/FootballTickets.Infrastructure
dotnet add tests/FootballTickets.Application.Tests reference
src/FootballTickets.Application
dotnet add tests/FootballTickets.Application.Tests reference
src/FootballTickets.Domain
dotnet add tests/FootballTickets.Api.IntegrationTests reference
src/FootballTickets.Api
```

Проверка: `dotnet build` в корне должен пройти без ошибок. Затем коммит:

```
git add .
git commit -m "chore: create solution with layered projects"
```

Шаг 4. Domain: сущности

В проекте Domain создайте папки `Entities` и `Enums`. Каждый класс в отдельном файле, ниже они собраны вместе для краткости.

Enums:

```
namespace FootballTickets.Domain.Enums;

public enum UserRole { User = 0, Admin = 1 }
public enum MatchStatus { Scheduled = 0, Cancelled = 1, Finished = 2 }
public enum OrderStatus { Paid = 0, Cancelled = 1 }
```

Entities:

```
using FootballTickets.Domain.Enums;
namespace FootballTickets.Domain.Entities;

public class User
{
    public Guid Id { get; set; }
    public string Email { get; set; } = null!;
    public string PasswordHash { get; set; } = null!;
    public string FullName { get; set; } = null!;
    public UserRole Role { get; set; } = UserRole.User;
```

```
    public DateTime CreatedAt { get; set; }
    public List<Order> Orders { get; set; } = new();
}

public class Team
{
    public Guid Id { get; set; }
    public string Name { get; set; } = null!;
    public string City { get; set; } = null!;
}

public class Stadium
{
    public Guid Id { get; set; }
    public string Name { get; set; } = null!;
    public string City { get; set; } = null!;
    public List<Sector> Sectors { get; set; } = new();
}

public class Sector
{
    public Guid Id { get; set; }
    public Guid StadiumId { get; set; }
    public Stadium Stadium { get; set; } = null!;
    public string Name { get; set; } = null!;
    public decimal PriceMultiplier { get; set; } = 1.0m;
    public List<Seat> Seats { get; set; } = new();
}

public class Seat
{
    public Guid Id { get; set; }
    public Guid SectorId { get; set; }
    public Sector Sector { get; set; } = null!;
    public int Row { get; set; }
    public int Number { get; set; }
}

public class Match
{
    public Guid Id { get; set; }
    public Guid HomeTeamId { get; set; }
    public Team HomeTeam { get; set; } = null!;
    public Guid AwayTeamId { get; set; }
    public Team AwayTeam { get; set; } = null!;
    public Guid StadiumId { get; set; }
```

```
    public Stadium Stadium { get; set; } = null!;
    public DateTime StartsAt { get; set; }
    public decimal BasePrice { get; set; }
    public MatchStatus Status { get; set; } = MatchStatus.Scheduled;
}

public class Order
{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;
    public DateTime CreatedAt { get; set; }
    public OrderStatus Status { get; set; } = OrderStatus.Paid;
    public decimal TotalAmount { get; set; }
    public List<Ticket> Tickets { get; set; } = new();
}

public class Ticket
{
    public Guid Id { get; set; }
    public Guid OrderId { get; set; }
    public Order Order { get; set; } = null!;
    public Guid MatchId { get; set; }
    public Match Match { get; set; } = null!;
    public Guid SeatId { get; set; }
    public Seat Seat { get; set; } = null!;
    public decimal Price { get; set; }
    public bool IsCancelled { get; set; }
}
```

Зачем так: сущности без логики и без зависимостей от EF, поэтому Domain остается "чистым". Правила (лимит мест, проверка времени) живут в сервисах Application. Когда освоитесь, часть правил можно вынести внутрь сущностей, это будет хорошим пунктом в разделе "Что бы улучшил".

Коммит:

```
git add .
git commit -m "feat(domain): add entities and enums"
```

Шаг 5. PostgreSQL в Docker

В корне репозитория создайте файл `docker-compose.yml`:

```
services:
  db:
    image: postgres:17
    container_name: tickets-db
    environment:
      POSTGRES_DB: football_tickets
      POSTGRES_USER: tickets
      POSTGRES_PASSWORD: tickets_dev_pass
    ports:
      - "5432:5432"
    volumes:
      - tickets_pgdata:/var/lib/postgresql/data

volumes:
  tickets_pgdata:
```

Запуск и проверка:

```
docker compose up -d db
docker ps
```

В DBeaver создайте подключение: хост localhost, порт 5432, база football_tickets, пользователь tickets, пароль tickets_dev_pass. Если подключилось, база работает. Пароль здесь только для локальной разработки, поэтому его можно держать в compose-файле, а в README честно написать, что это dev-значение.

Строку подключения и ключ JWT храним в user-secrets, чтобы они не попали в Git:

```
cd src/FootballTickets.Api
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:Default"
"Host=localhost;Port=5432;Database=football_tickets;Username=tickets;Password
=tickets_dev_pass"
dotnet user-secrets set "Jwt:Key" "придумайте-длинную-случайную-строку-не-
короче-32-символов"
cd ../..
```

В `appsettings.json` Api добавьте несекретные настройки JWT:

```
"Jwt": {
  "Issuer": "FootballTickets",
  "Audience": "FootballTicketsClients",
  "ExpiresMinutes": 60
}
```

Коммит:

```
git add .
git commit -m "chore: add postgres docker-compose and jwt settings"
```

Шаг 6. Application: контракты

Пакет и интерфейс доступа к БД:

```
dotnet add src/FootballTickets.Application package
Microsoft.EntityFrameworkCore
```

Файл `Application/Common/IAppDbContext.cs`:

```
using FootballTickets.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace FootballTickets.Application.Common;

public interface IAppDbContext
{
    DbSet<User> Users { get; }
    DbSet<Team> Teams { get; }
    DbSet<Stadium> Stadiums { get; }
    DbSet<Sector> Sectors { get; }
    DbSet<Seat> Seats { get; }
    DbSet<Match> Matches { get; }
    DbSet<Order> Orders { get; }
    DbSet<Ticket> Tickets { get; }
    Task<int> SaveChangesAsync(CancellationToken ct = default);
}
```

И два интерфейса для безопасности (нужны и сидеру, и AuthService):

```
using FootballTickets.Domain.Entities;
namespace FootballTickets.Application.Common;

public interface IPasswordService
{
    string Hash(string password);
    bool Verify(string hash, string password);
}

public interface IJwtTokenGenerator
{
    string Generate(User user);
}
```

Шаг 7. Infrastructure: DbContext и конфигурации таблиц

Пакеты:

```
dotnet add src/FootballTickets.Infrastructure package
Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add src/FootballTickets.Infrastructure package
Microsoft.EntityFrameworkCore.Design
dotnet add src/FootballTickets.Infrastructure package
Microsoft.Extensions.Identity.Core
dotnet add src/FootballTickets.Infrastructure package
Microsoft.Extensions.Configuration.Abstractions
dotnet add src/FootballTickets.Api package
Microsoft.EntityFrameworkCore.Design
```

Файл `Infrastructure/Persistence/AppDbContext.cs`:

```
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace FootballTickets.Infrastructure.Persistence;

public class AppDbContext : DbContext, IAppDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<Team> Teams => Set<Team>();
    public DbSet<Stadium> Stadiums => Set<Stadium>();
```

```
    public DbSet<Sector> Sectors => Set<Sector>();
    public DbSet<Seat> Seats => Set<Seat>();
    public DbSet<Match> Matches => Set<Match>();
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<Ticket> Tickets => Set<Ticket>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {

modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

Конфигурации таблиц лежат в `Infrastructure/Persistence/Configurations/`, по файлу на сущность. Это держит Domain чистым (никаких атрибутов EF в сущностях). Пользователь и команда:

```
using FootballTickets.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace FootballTickets.Infrastructure.Persistence.Configurations;

public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.Email).HasMaxLength(256).IsRequired();
        b.HasIndex(x => x.Email).IsUnique();
        b.Property(x => x.PasswordHash).IsRequired();
        b.Property(x => x.FullName).HasMaxLength(200).IsRequired();
        b.Property(x => x.Role).HasConversion<string>().HasMaxLength(20);
    }
}

public class TeamConfiguration : IEntityTypeConfiguration<Team>
{
    public void Configure(EntityTypeBuilder<Team> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.Name).HasMaxLength(100).IsRequired();
```

```
        b.HasIndex(x => x.Name).IsUnique();
        b.Property(x => x.City).HasMaxLength(100).IsRequired();
    }
}
```

Стадион, сектор, место:

```
public class StadiumConfiguration : IEntityTypeConfiguration<Stadium>
{
    public void Configure(EntityTypeBuilder<Stadium> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.Name).HasMaxLength(150).IsRequired();
        b.Property(x => x.City).HasMaxLength(100).IsRequired();
    }
}

public class SectorConfiguration : IEntityTypeConfiguration<Sector>
{
    public void Configure(EntityTypeBuilder<Sector> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.Name).HasMaxLength(50).IsRequired();
        b.Property(x => x.PriceMultiplier).HasPrecision(5, 2);
        b.HasOne(x => x.Stadium).WithMany(s => s.Sectors)
            .HasForeignKey(x =>
x.StadiumId).OnDelete(DeleteBehavior.Cascade);
    }
}

public class SeatConfiguration : IEntityTypeConfiguration<Seat>
{
    public void Configure(EntityTypeBuilder<Seat> b)
    {
        b.HasKey(x => x.Id);
        b.HasIndex(x => new { x.SectorId, x.Row, x.Number }).IsUnique();
        b.HasOne(x => x.Sector).WithMany(s => s.Seats)
            .HasForeignKey(x => x.SectorId).OnDelete(DeleteBehavior.Cascade);
    }
}
```

Матч (две связи с командой и проверка, что команды разные):

```
public class MatchConfiguration : IEntityTypeConfiguration<Match>
{
    public void Configure(EntityTypeBuilder<Match> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.BasePrice).HasPrecision(10, 2);
        b.Property(x => x.Status).HasConversion<string>().HasMaxLength(20);
        b.HasIndex(x => x.StartsAt);

        b.HasOne(x => x.HomeTeam).WithMany()
            .HasForeignKey(x =>
x.HomeTeamId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne(x => x.AwayTeam).WithMany()
            .HasForeignKey(x =>
x.AwayTeamId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne(x => x.Stadium).WithMany()
            .HasForeignKey(x =>
x.StadiumId).OnDelete(DeleteBehavior.Restrict);

        b.ToTable(t => t.HasCheckConstraint(
            "CK_Matches_DifferentTeams", "\"HomeTeamId\" <>
\"AwayTeamId\""));
    }
}
```

Заказ и билет. Здесь главная фишка проекта, частичный уникальный индекс: одно место на матч может быть занято только одним неотмененным билетом.

```
public class OrderConfiguration : IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.TotalAmount).HasPrecision(10, 2);
        b.Property(x => x.Status).HasConversion<string>().HasMaxLength(20);
        b.HasOne(x => x.User).WithMany(u => u.Orders)
            .HasForeignKey(x => x.UserId).OnDelete(DeleteBehavior.Restrict);
    }
}

public class TicketConfiguration : IEntityTypeConfiguration<Ticket>
{
    public void Configure(EntityTypeBuilder<Ticket> b)
    {
        b.HasKey(x => x.Id);
        b.Property(x => x.Price).HasPrecision(10, 2);
```

```
        b.HasOne(x => x.Order).WithMany(o => o.Tickets)
            .HasForeignKey(x => x.OrderId).OnDelete(DeleteBehavior.Cascade);
        b.HasOne(x => x.Match).WithMany()
            .HasForeignKey(x => x.MatchId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne(x => x.Seat).WithMany()
            .HasForeignKey(x => x.SeatId).OnDelete(DeleteBehavior.Restrict);

        b.HasIndex(x => new { x.MatchId, x.SeatId })
            .IsUnique()
            .HasFilter("\"IsCancelled\" = false");
    }
}
```

Для собеседования: если двое одновременно купят одно место, второй `SaveChanges` упадет с `DbUpdateException` из-за индекса. Мы поймаем его в OrderService и вернем 409 Conflict. Это правильнее, чем проверка "свободно ли место" в коде: проверка без блокировки не защищает от гонки.

Регистрация инфраструктуры. Файл `Infrastructure/DependencyInjection.cs` (реализацию PasswordService и JwtTokenGenerator добавим ниже, пока подключаем только БД и пароли):

```
using FootballTickets.Application.Common;
using FootballTickets.Infrastructure.Persistence;
using FootballTickets.Infrastructure.Security;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace FootballTickets.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services, IConfiguration config)
    {
        services.AddDbContext<AppDbContext>(o =>
            o.UseNpgsql(config.GetConnectionString("Default")));
        services.AddScoped<IAppDbContext>(sp =>
sp.GetRequiredService<AppDbContext>());
        services.AddSingleton<IPasswordService, PasswordService>();
        return services;
    }
}
```

Файл `Infrastructure/Security/PasswordService.cs`:

```
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using Microsoft.AspNetCore.Identity;

namespace FootballTickets.Infrastructure.Security;

public class PasswordService : IPasswordService
{
    private readonly PasswordHasher<User> _hasher = new();

    public string Hash(string password) => _hasher.HashPassword(null!,
password);

    public bool Verify(string hash, string password) =>
        _hasher.VerifyHashedPassword(null!, hash, password) !=
PasswordVerificationResult.Failed;
}
```

В `Api/Program.cs` перед `builder.Build()` добавьте:

```
builder.Services.AddInfrastructure(builder.Configuration);
```

Если среда подчеркивает `PasswordHasher`, проверьте, что установлен пакет Microsoft.Extensions.Identity.Core: пространство имен у него `Microsoft.AspNetCore.Identity`, это нормально.

Проверка: `dotnet build`. Коммит:

```
git add .
git commit -m "feat(infra): add DbContext, entity configurations and password
service"
```

Шаг 8. Первая миграция

```
dotnet ef migrations add InitialCreate --project
src/FootballTickets.Infrastructure --startup-project src/FootballTickets.Api
--output-dir Persistence/Migrations
dotnet ef database update --project src/FootballTickets.Infrastructure --
startup-project src/FootballTickets.Api
```

Что происходит: первая команда сравнивает модель в коде с пустой БД и генерирует C#-файл миграции, вторая применяет его к базе из строки подключения (она берется из user-secrets проекта Api).

Откройте DBeaver, обновите схему public и убедитесь, что есть 8 таблиц плюс служебная `__EFMigrationsHistory`. Откройте индексы таблицы Tickets: у индекса по (MatchId, SeatId) должно быть условие `WHERE "IsCancelled" = false`. Откройте ограничения таблицы Matches: там должна быть проверка CK_Matches_DifferentTeams.

Если получили ошибку "Unable to create a DbContext": проверьте, что запущен контейнер с базой, что `AddInfrastructure` вызван в Program.cs и что секреты заданы (`dotnet user-secrets list` в папке Api).

Коммит:

```
git add .
git commit -m "feat(infra): add initial migration"
```

Шаг 9. Сидер тестовых данных

Файл `Infrastructure/Persistence/DbSeeder.cs`. Он создает 4 команды, один стадион с тремя секторами (210 мест), три будущих матча и двух пользователей:

```
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using FootballTickets.Domain.Enums;
using Microsoft.EntityFrameworkCore;

namespace FootballTickets.Infrastructure.Persistence;

public static class DbSeeder
{
    public static async Task SeedAsync(AppDbContext db, IPasswordService
passwords)
    {
        if (await db.Teams.AnyAsync()) return;

        var teams = new[]
        {
            new Team { Id = Guid.NewGuid(), Name = "Динамо Минск", City =
"Минск" },
            new Team { Id = Guid.NewGuid(), Name = "БАТЭ Борисов", City =
"Борисов" },
            new Team { Id = Guid.NewGuid(), Name = "Шахтер Солигорск", City =
"Солигорск" },
            new Team { Id = Guid.NewGuid(), Name = "Динамо Брест", City =
"Брест" },
```

```
        };
        db.Teams.AddRange(teams);

        var stadium = new Stadium { Id = Guid.NewGuid(), Name = "Стадион
Динамо", City = "Минск" };
        var sectorDefs = new[]
        {
            (Name: "VIP", Multiplier: 2.0m, Rows: 3, PerRow: 10),
            (Name: "A", Multiplier: 1.5m, Rows: 5, PerRow: 12),
            (Name: "B", Multiplier: 1.0m, Rows: 8, PerRow: 15),
        };
        foreach (var def in sectorDefs)
        {
            var sector = new Sector
            {
                Id = Guid.NewGuid(), Name = def.Name, PriceMultiplier =
def.Multiplier
            };
            for (var row = 1; row <= def.Rows; row++)
                for (var number = 1; number <= def.PerRow; number++)
                    sector.Seats.Add(new Seat { Id = Guid.NewGuid(), Row =
row, Number = number });
            stadium.Sectors.Add(sector);
        }
        db.Stadiums.Add(stadium);

        db.Matches.AddRange(
            new Match { Id = Guid.NewGuid(), HomeTeamId = teams[0].Id,
AwayTeamId = teams[1].Id,
                StadiumId = stadium.Id, StartsAt =
DateTime.UtcNow.AddDays(7), BasePrice = 20m },
            new Match { Id = Guid.NewGuid(), HomeTeamId = teams[0].Id,
AwayTeamId = teams[2].Id,
                StadiumId = stadium.Id, StartsAt =
DateTime.UtcNow.AddDays(14), BasePrice = 25m },
            new Match { Id = Guid.NewGuid(), HomeTeamId = teams[3].Id,
AwayTeamId = teams[0].Id,
                StadiumId = stadium.Id, StartsAt =
DateTime.UtcNow.AddDays(21), BasePrice = 15m });

        db.Users.AddRange(
            new User { Id = Guid.NewGuid(), Email = "admin@tickets.local",
FullName = "Admin",
                Role = UserRole.Admin, PasswordHash =
passwords.Hash("Admin12345!"),
                CreatedAt = DateTime.UtcNow },
```

```
            new User { Id = Guid.NewGuid(), Email = "user@tickets.local",
FullName = "Test User",
                Role = UserRole.User, PasswordHash =
passwords.Hash("User12345!"),
                CreatedAt = DateTime.UtcNow });

        await db.SaveChangesAsync();
    }
}
```

Запуск миграций и сидера при старте в режиме разработки. В `Api/Program.cs` после `var app = builder.Build();`:

```
if (app.Environment.IsDevelopment())
{
    using var scope = app.Services.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await db.Database.MigrateAsync();
    await DbSeeder.SeedAsync(db,
scope.ServiceProvider.GetRequiredService<IPasswordService>());
}
```

(нужны `using` для `FootballTickets.Infrastructure.Persistence`, `FootballTickets.Application.Common` и `Microsoft.EntityFrameworkCore`).

Проверка: запустите Api (`dotnet run --project src/FootballTickets.Api`), затем в DBeaver выполните:

```
SELECT count(*) FROM "Seats";      -- 210
SELECT count(*) FROM "Matches";    -- 3
SELECT "Email", "Role" FROM "Users";
```

Если цифры совпали, фундамент готов. Тестовые логины и пароли (admin@tickets.local, user@tickets.local) потом укажите в README с пометкой, что это только для демонстрации.

Коммит:

```
git add .
git commit -m "feat(infra): add dev data seeder and auto-migration in
development"
git push
```

Шаг 10. Единая обработка ошибок и валидация

При чтении кода на собеседовании первым делом смотрят, как устроены ошибки. Делаем так: сервисы кидают свои исключения, а один глобальный обработчик превращает их в ответы ProblemDetails с нужным кодом.

Файл `Application/Common/Exceptions.cs`:

```
namespace FootballTickets.Application.Common;

public abstract class AppException : Exception
{
    protected AppException(string message) : base(message) { }
}

public class NotFoundException : AppException { public
NotFoundException(string m) : base(m) { } }
public class ConflictException : AppException { public
ConflictException(string m) : base(m) { } }
public class ForbiddenException : AppException { public
ForbiddenException(string m) : base(m) { } }
public class UnauthorizedException : AppException { public
UnauthorizedException(string m) : base(m) { } }
public class BusinessRuleException : AppException { public
BusinessRuleException(string m) : base(m) { } }
```

Пакеты и регистрация сервисов Application:

```
dotnet add src/FootballTickets.Application package FluentValidation
dotnet add src/FootballTickets.Application package
FluentValidation.DependencyInjectionExtensions
```

```
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;

namespace FootballTickets.Application;

public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection
services)
    {

services.AddValidatorsFromAssembly(typeof(DependencyInjection).Assembly);
        services.AddScoped<IAuthService, AuthService>();
        services.AddScoped<IMatchService, MatchService>();
        // IOrderService добавим на шаге 13
        return services;
    }
}
```

Глобальный обработчик в Api, файл `Api/Middleware/GlobalExceptionHandler.cs`:

```
using FluentValidation;
using FootballTickets.Application.Common;
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;

namespace FootballTickets.Api.Middleware;

public class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;

    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) =>
_logger = logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext context, Exception ex, CancellationToken ct)
    {
        var (status, title) = ex switch
        {
            NotFoundException => (404, "Not found"),
            ConflictException => (409, "Conflict"),
            ForbiddenException => (403, "Forbidden"),
            UnauthorizedException => (401, "Unauthorized"),
            BusinessRuleException => (400, "Business rule violated"),
            ValidationException => (400, "Validation failed"),
```

```
            _ => (500, "Server error")
        };

        if (status == 500) _logger.LogError(ex, "Unhandled exception");

        var problem = new ProblemDetails
        {
            Status = status,
            Title = title,
            Detail = status == 500 ? "Unexpected error" : ex.Message
        };
        if (ex is ValidationException ve)
        {
            problem.Extensions["errors"] = ve.Errors
                .GroupBy(e => e.PropertyName)
                .ToDictionary(g => g.Key, g => g.Select(e =>
e.ErrorMessage).ToArray());
        }

        context.Response.StatusCode = status;
        await context.Response.WriteAsJsonAsync(problem, ct);
        return true;
    }
}
```

В `Program.cs`:

```
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
builder.Services.AddApplication();
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUser, CurrentUser>();

// после var app = builder.Build();
app.UseExceptionHandler();
```

Интерфейс текущего пользователя (нужен сервисам заказов), `Application/Common/ICurrentUser.cs`:

```
namespace FootballTickets.Application.Common;

public interface ICurrentUser
{
    Guid UserId { get; }
}
```

Реализация, `Api/Services/CurrentUser.cs`:

```
using FootballTickets.Application.Common;

namespace FootballTickets.Api.Services;

public class CurrentUser : ICurrentUser
{
    private readonly IHttpContextAccessor _http;
    public CurrentUser(IHttpContextAccessor http) => _http = http;

    public Guid UserId
    {
        get
        {
            var sub = _http.HttpContext?.User.FindFirst("sub")?.Value;
            return Guid.TryParse(sub, out var id)
                ? id
                : throw new UnauthorizedException("Not authenticated");
        }
    }
}
```

Коммит: `feat(api): add global exception handling and current user`.

Шаг 11. Регистрация, вход и JWT

DTO и валидаторы (`Application/Auth/`):

```
using FluentValidation;

namespace FootballTickets.Application.Auth;

public record RegisterRequest(string Email, string Password, string
FullName);
public record LoginRequest(string Email, string Password);
public record AuthResponse(string Token);

public class RegisterRequestValidator : AbstractValidator<RegisterRequest>
{
    public RegisterRequestValidator()
    {
        RuleFor(x => x.Email).NotEmpty().EmailAddress().MaximumLength(256);
        RuleFor(x => x.Password).NotEmpty().MinimumLength(8);
        RuleFor(x => x.FullName).NotEmpty().MaximumLength(200);
    }
```

```
}

public class LoginRequestValidator : AbstractValidator<LoginRequest>
{
    public LoginRequestValidator()
    {
        RuleFor(x => x.Email).NotEmpty();
        RuleFor(x => x.Password).NotEmpty();
    }
}
```

Сервис (`Application/Auth/AuthService.cs`):

```
using FluentValidation;
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using FootballTickets.Domain.Enums;
using Microsoft.EntityFrameworkCore;

namespace FootballTickets.Application.Auth;

public interface IAuthService
{
    Task<AuthResponse> RegisterAsync(RegisterRequest request,
CancellationToken ct);
    Task<AuthResponse> LoginAsync(LoginRequest request, CancellationToken
ct);
}

public class AuthService : IAuthService
{
    private readonly IAppDbContext _db;
    private readonly IPasswordService _passwords;
    private readonly IJwtTokenGenerator _jwt;
    private readonly IValidator<RegisterRequest> _registerValidator;
    private readonly IValidator<LoginRequest> _loginValidator;

    public AuthService(IAppDbContext db, IPasswordService passwords,
IJwtTokenGenerator jwt,
        IValidator<RegisterRequest> registerValidator,
IValidator<LoginRequest> loginValidator)
    {
        _db = db; _passwords = passwords; _jwt = jwt;
        _registerValidator = registerValidator; _loginValidator =
loginValidator;
    }
```

```
    public async Task<AuthResponse> RegisterAsync(RegisterRequest request,
CancellationToken ct)
    {
        await _registerValidator.ValidateAndThrowAsync(request, ct);
        var email = request.Email.Trim().ToLowerInvariant();

        if (await _db.Users.AnyAsync(u => u.Email == email, ct))
            throw new ConflictException("Email is already registered");

        var user = new User
        {
            Id = Guid.NewGuid(),
            Email = email,
            FullName = request.FullName.Trim(),
            PasswordHash = _passwords.Hash(request.Password),
            Role = UserRole.User,
            CreatedAt = DateTime.UtcNow
        };
        _db.Users.Add(user);
        await _db.SaveChangesAsync(ct);
        return new AuthResponse(_jwt.Generate(user));
    }

    public async Task<AuthResponse> LoginAsync(LoginRequest request,
CancellationToken ct)
    {
        await _loginValidator.ValidateAndThrowAsync(request, ct);
        var email = request.Email.Trim().ToLowerInvariant();

        var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == email,
ct);
        if (user is null || !_passwords.Verify(user.PasswordHash,
request.Password))
            throw new UnauthorizedException("Invalid email or password");

        return new AuthResponse(_jwt.Generate(user));
    }
}
```

Обратите внимание: на неверный email и неверный пароль отвечаем одним сообщением, чтобы нельзя было перебором выяснить, какие email зарегистрированы. Роль при регистрации всегда User, админа создаем только сидером или напрямую в БД.

Генератор токена. Пакет и класс в Infrastructure:

```
dotnet add src/FootballTickets.Infrastructure package
System.IdentityModel.Tokens.Jwt
```

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;

namespace FootballTickets.Infrastructure.Security;

public class JwtOptions
{
    public string Issuer { get; set; } = "";
    public string Audience { get; set; } = "";
    public string Key { get; set; } = "";
    public int ExpiresMinutes { get; set; } = 60;
}

public class JwtTokenGenerator : IJwtTokenGenerator
{
    private readonly JwtOptions _options;
    public JwtTokenGenerator(IOptions<JwtOptions> options) => _options =
options.Value;

    public string Generate(User user)
    {
        var claims = new[]
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new Claim(JwtRegisteredClaimNames.Email, user.Email),
            new Claim("role", user.Role.ToString())
        };
        var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_options.Key));
        var credentials = new SigningCredentials(key,
SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _options.Issuer,
            audience: _options.Audience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(_options.ExpiresMinutes),
```

```
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

В `AddInfrastructure` добавьте строку:
`services.AddSingleton<IJwtTokenGenerator, JwtTokenGenerator>();`

Роль кладем в claim с коротким именем "role", а id в "sub". Это важно: в новых версиях ASP.NET Core входящие claims не переименовываются, и если оставить длинные URI-имена, атрибут `[Authorize(Roles = "Admin")]` может молча не работать.

Подключение аутентификации в Api.

```
dotnet add src/FootballTickets.Api package
Microsoft.AspNetCore.Authentication.JwtBearer
```

В `Program.cs`:

```
builder.Services.Configure<JwtOptions>
(builder.Configuration.GetSection("Jwt"));
var jwt = builder.Configuration.GetSection("Jwt").Get<JwtOptions>()!;

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.MapInboundClaims = false;
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true, ValidIssuer = jwt.Issuer,
            ValidateAudience = true, ValidAudience = jwt.Audience,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.Key)),
            RoleClaimType = "role",
            NameClaimType = "sub"
        };
    });
builder.Services.AddAuthorization();
```

```
// после var app = builder.Build(); и UseExceptionHandler:
app.UseAuthentication();
app.UseAuthorization();
```

Порядок важен: сначала `UseAuthentication`, потом `UseAuthorization`, и оба до `MapControllers`.

Контроллер `Api/Controllers/AuthController.cs`:

```
using FootballTickets.Application.Auth;
using Microsoft.AspNetCore.Mvc;

namespace FootballTickets.Api.Controllers;

[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _auth;
    public AuthController(IAuthService auth) => _auth = auth;

    [HttpPost("register")]
    public async Task<ActionResult<AuthResponse>> Register(RegisterRequest
request, CancellationToken ct)
        => Ok(await _auth.RegisterAsync(request, ct));

    [HttpPost("login")]
    public async Task<ActionResult<AuthResponse>> Login(LoginRequest request,
CancellationToken ct)
        => Ok(await _auth.LoginAsync(request, ct));
}
```

Документация API. Шаблон уже подключает OpenAPI. Для удобного интерфейса:

```
dotnet add src/FootballTickets.Api package Scalar.AspNetCore
```

И в `Program.cs` внутри блока `if (app.Environment.IsDevelopment())` рядом с `app.MapOpenApi();` добавьте `app.MapScalarApiReference();` (`using Scalar.AspNetCore;`). Интерфейс откроется по адресу /scalar/v1. Запросы с токеном удобнее слать из Postman или из файла `.http` в Visual Studio, заголовок `Authorization: Bearer <токен>`.

Проверка:

1. Запустите Api и отправьте `POST /api/auth/login` с телом `{"email":"user@tickets.local","password":"User12345!"}`. Должен прийти токен.
2. Вставьте токен на сайте jwt.io и убедитесь, что внутри sub, email и role.
3. Попробуйте зарегистрироваться с коротким паролем: должен прийти 400 с описанием ошибок в `errors`.
4. Попробуйте зарегистрироваться на уже существующий email: 409.

Коммит: `feat(auth): add registration, login and jwt authentication`.

Шаг 12. Матчи: чтение и создание

DTO (`Application/Matches/Dtos.cs`):

```
namespace FootballTickets.Application.Matches;

public record TeamDto(Guid Id, string Name, string City);
public record MatchDto(Guid Id, TeamDto HomeTeam, TeamDto AwayTeam, string
Stadium,
    DateTime StartsAt, decimal BasePrice, string Status);
public record MatchFilter(DateTime? From, DateTime? To, Guid? TeamId, int
Page = 1, int PageSize = 20);
public record PagedResult<T>(IReadOnlyList<T> Items, int Page, int PageSize,
int TotalCount);
public record CreateMatchRequest(Guid HomeTeamId, Guid AwayTeamId, Guid
StadiumId,
    DateTime StartsAt, decimal BasePrice);
public record SeatDto(Guid Id, int Row, int Number, decimal Price, bool
IsTaken);
public record SectorSeatsDto(Guid SectorId, string Sector,
IReadOnlyList<SeatDto> Seats);
```

Валидатор создания матча:

```
using FluentValidation;
namespace FootballTickets.Application.Matches;

public class CreateMatchRequestValidator :
AbstractValidator<CreateMatchRequest>
{
    public CreateMatchRequestValidator()
    {
        RuleFor(x => x.HomeTeamId).NotEmpty();
        RuleFor(x => x.AwayTeamId).NotEmpty().NotEqual(x => x.HomeTeamId)
            .WithMessage("Teams must be different");
        RuleFor(x => x.StadiumId).NotEmpty();
        RuleFor(x => x.StartsAt).Must(d => d.ToUniversalTime() >
DateTime.UtcNow)
            .WithMessage("Match must start in the future");
        RuleFor(x => x.BasePrice).GreaterThan(0);
    }
}
```

Важная ловушка с датами. Npgsql не принимает `DateTime` с `Kind = Unspecified` для столбца timestamptz (получите исключение при запросе вида `?from=2026-11-01`). Добавьте общий помощник в Application/Common и применяйте ко всем датам, пришедшим снаружи:

```
namespace FootballTickets.Application.Common;

public static class DateTimeExtensions
{
    public static DateTime ToUtc(this DateTime value) => value.Kind switch
    {
        DateTimeKind.Utc => value,
        DateTimeKind.Local => value.ToUniversalTime(),
        _ => DateTime.SpecifyKind(value, DateTimeKind.Utc)
    };
}
```

Сервис (`Application/Matches/MatchService.cs`). Показываю ключевые методы, остальные пишите по тому же образцу:

```
using FluentValidation;
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace FootballTickets.Application.Matches;
```

```
public interface IMatchService
{
    Task<PagedResult<MatchDto>> GetListAsync(MatchFilter filter,
CancellationToken ct);
    Task<MatchDto> GetByIdAsync(Guid id, CancellationToken ct);
    Task<List<SectorSeatsDto>> GetSeatsAsync(Guid matchId, CancellationToken
ct);
    Task<MatchDto> CreateAsync(CreateMatchRequest request, CancellationToken
ct);
    Task DeleteAsync(Guid id, CancellationToken ct);
}

public class MatchService : IMatchService
{
    private readonly IAppDbContext _db;
    private readonly IValidator<CreateMatchRequest> _createValidator;

    public MatchService(IAppDbContext db, IValidator<CreateMatchRequest>
createValidator)
    {
        _db = db; _createValidator = createValidator;
    }

    public async Task<PagedResult<MatchDto>> GetListAsync(MatchFilter f,
CancellationToken ct)
    {
        var page = Math.Max(f.Page, 1);
        var size = Math.Clamp(f.PageSize, 1, 100);

        var query = _db.Matches.AsNoTracking().AsQueryable();
        if (f.From is not null) { var from = f.From.Value.ToUtc(); query =
query.Where(m => m.StartsAt >= from); }
        if (f.To is not null) { var to = f.To.Value.ToUtc(); query =
query.Where(m => m.StartsAt <= to); }
        if (f.TeamId is not null)
            query = query.Where(m => m.HomeTeamId == f.TeamId || m.AwayTeamId
== f.TeamId);

        var total = await query.CountAsync(ct);
        var items = await query
            .OrderBy(m => m.StartsAt)
            .Skip((page - 1) * size).Take(size)
            .Select(m => new MatchDto(m.Id,
                new TeamDto(m.HomeTeam.Id, m.HomeTeam.Name, m.HomeTeam.City),
                new TeamDto(m.AwayTeam.Id, m.AwayTeam.Name, m.AwayTeam.City),
```

```
                m.Stadium.Name, m.StartsAt, m.BasePrice,
m.Status.ToString()))
            .ToListAsync(ct);

        return new PagedResult<MatchDto>(items, page, size, total);
    }

    public async Task<MatchDto> GetByIdAsync(Guid id, CancellationToken ct)
    {
        return await _db.Matches.AsNoTracking()
            .Where(m => m.Id == id)
            .Select(m => new MatchDto(m.Id,
                new TeamDto(m.HomeTeam.Id, m.HomeTeam.Name, m.HomeTeam.City),
                new TeamDto(m.AwayTeam.Id, m.AwayTeam.Name, m.AwayTeam.City),
                m.Stadium.Name, m.StartsAt, m.BasePrice,
m.Status.ToString()))
            .FirstOrDefaultAsync(ct)
            ?? throw new NotFoundException("Match not found");
    }

    public async Task<List<SectorSeatsDto>> GetSeatsAsync(Guid matchId,
CancellationToken ct)
    {
        var match = await _db.Matches.AsNoTracking().FirstOrDefaultAsync(m =>
m.Id == matchId, ct)
            ?? throw new NotFoundException("Match not found");

        var taken = (await _db.Tickets.AsNoTracking()
            .Where(t => t.MatchId == matchId && !t.IsCancelled)
            .Select(t => t.SeatId).ToListAsync(ct)).ToHashSet();

        var sectors = await _db.Sectors.AsNoTracking()
            .Where(s => s.StadiumId == match.StadiumId)
            .Include(s => s.Seats)
            .OrderBy(s => s.Name)
            .ToListAsync(ct);

        return sectors.Select(s => new SectorSeatsDto(s.Id, s.Name,
            s.Seats.OrderBy(x => x.Row).ThenBy(x => x.Number)
                .Select(x => new SeatDto(x.Id, x.Row, x.Number,
                    Math.Round(match.BasePrice * s.PriceMultiplier, 2),
taken.Contains(x.Id)))
                .ToList())).ToList();
    }

    public async Task<MatchDto> CreateAsync(CreateMatchRequest r,
```

```
CancellationToken ct)
    {
        await _createValidator.ValidateAndThrowAsync(r, ct);

        if (!await _db.Teams.AnyAsync(t => t.Id == r.HomeTeamId, ct))
            throw new NotFoundException("Home team not found");
        if (!await _db.Teams.AnyAsync(t => t.Id == r.AwayTeamId, ct))
            throw new NotFoundException("Away team not found");
        if (!await _db.Stadiums.AnyAsync(s => s.Id == r.StadiumId, ct))
            throw new NotFoundException("Stadium not found");

        var match = new Match
        {
            Id = Guid.NewGuid(),
            HomeTeamId = r.HomeTeamId,
            AwayTeamId = r.AwayTeamId,
            StadiumId = r.StadiumId,
            StartsAt = r.StartsAt.ToUtc(),
            BasePrice = r.BasePrice
        };
        _db.Matches.Add(match);
        await _db.SaveChangesAsync(ct);
        return await GetByIdAsync(match.Id, ct);
    }

    public async Task DeleteAsync(Guid id, CancellationToken ct)
    {
        var match = await _db.Matches.FirstOrDefaultAsync(m => m.Id == id,
ct)
            ?? throw new NotFoundException("Match not found");

        if (await _db.Tickets.AnyAsync(t => t.MatchId == id &&
!t.IsCancelled, ct))
            throw new ConflictException("Match has active tickets and cannot
be deleted");

        _db.Matches.Remove(match);
        await _db.SaveChangesAsync(ct);
    }
}
```

У метода `DeleteAsync` есть тонкость: если у матча остались только отмененные билеты, строка в Tickets все равно ссылается на матч, и из-за `DeleteBehavior.Restrict` удаление упадет на уровне БД. Подумайте, как это

решить (например, запретить удаление при любых билетах или помечать матч как Cancelled вместо удаления). Это хороший вопрос для раздела "Что бы улучшил" и для собеседования.

Контроллер `Api/Controllers/MatchesController.cs`:

```
using FootballTickets.Application.Matches;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace FootballTickets.Api.Controllers;

[ApiController]
[Route("api/matches")]
public class MatchesController : ControllerBase
{
    private readonly IMatchService _matches;
    public MatchesController(IMatchService matches) => _matches = matches;

    [HttpGet]
    public async Task<ActionResult<PagedResult<MatchDto>>> GetList(
        [FromQuery] MatchFilter filter, CancellationToken ct)
        => Ok(await _matches.GetListAsync(filter, ct));

    [HttpGet("{id:guid}")]
    public async Task<ActionResult<MatchDto>> GetById(Guid id,
CancellationToken ct)
        => Ok(await _matches.GetByIdAsync(id, ct));

    [HttpGet("{id:guid}/seats")]
    public async Task<ActionResult<List<SectorSeatsDto>>> GetSeats(Guid id,
CancellationToken ct)
        => Ok(await _matches.GetSeatsAsync(id, ct));

    [Authorize(Roles = "Admin")]
    [HttpPost]
    public async Task<ActionResult<MatchDto>> Create(CreateMatchRequest
request, CancellationToken ct)
    {
        var created = await _matches.CreateAsync(request, ct);
        return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
    }

    [Authorize(Roles = "Admin")]
    [HttpDelete("{id:guid}")]
```

```
    public async Task<IActionResult> Delete(Guid id, CancellationToken ct)
    {
        await _matches.DeleteAsync(id, ct);
        return NoContent();
    }
}
```

Что дописать самостоятельно (по образцу выше, это ваша тренировка и материал для обсуждения на собеседовании):

- `PUT /api/matches/{id}`: изменение времени и цены, нельзя менять матч, который уже начался;
- `TeamsController` и `StadiumsController` (GET для всех, остальное только Admin);
- сборка списка команд и стадионов с сортировкой по названию.

Проверка: `GET /api/matches` возвращает 3 матча, `GET /api/matches/{id}/seats` возвращает 3 сектора и 210 мест с ценой (VIP дороже B вдвое), `POST /api/matches` без токена дает 401, с токеном обычного пользователя 403, с токеном admin 201.

Коммит: `feat(matches): add matches endpoints with filtering, paging and seat map`.

Шаг 13. Заказы: покупка и отмена (ядро проекта)

Этот шаг самый важный: здесь бизнес-правила и защита от двойной продажи места. Именно по нему вас будут расспрашивать.

DTO и валидатор (`Application/Orders/Dtos.cs`):

```
using FluentValidation;

namespace FootballTickets.Application.Orders;

public record CreateOrderRequest(Guid MatchId, List<Guid> SeatIds);

public record TicketDto(Guid Id, Guid SeatId, string Sector, int Row, int
Number,
    decimal Price, bool IsCancelled);

public record OrderDto(Guid Id, Guid MatchId, string Match, DateTime
MatchStartsAt,
    DateTime CreatedAt, string Status, decimal TotalAmount,
IReadOnlyList<TicketDto> Tickets);
```

```
public class CreateOrderRequestValidator :
AbstractValidator<CreateOrderRequest>
{
    public CreateOrderRequestValidator()
    {
        RuleFor(x => x.MatchId).NotEmpty();
        RuleFor(x => x.SeatIds).NotNull().Must(s => s != null && s.Count is
>= 1 and <= 6)
            .WithMessage("An order must contain from 1 to 6 seats");
        RuleFor(x => x.SeatIds).Must(s => s == null || s.Distinct().Count()
== s.Count)
            .WithMessage("Seats must not repeat");
    }
}
```

Сервис (`Application/Orders/OrderService.cs`). Время берем через `TimeProvider` (встроен в .NET 8+), чтобы в тестах можно было подставить фиктивные часы:

```
using FluentValidation;
using FootballTickets.Application.Common;
using FootballTickets.Domain.Entities;
using FootballTickets.Domain.Enums;
using Microsoft.EntityFrameworkCore;

namespace FootballTickets.Application.Orders;

public interface IOrderService
{
    Task<OrderDto> CreateAsync(CreateOrderRequest request, CancellationToken
ct);
    Task<List<OrderDto>> GetMyAsync(CancellationToken ct);
    Task<OrderDto> GetByIdAsync(Guid id, CancellationToken ct);
    Task<OrderDto> CancelAsync(Guid id, CancellationToken ct);
}

public class OrderService : IOrderService
{
    private readonly IAppDbContext _db;
    private readonly ICurrentUser _currentUser;
    private readonly IValidator<CreateOrderRequest> _validator;
    private readonly TimeProvider _time;

    public OrderService(IAppDbContext db, ICurrentUser currentUser,
        IValidator<CreateOrderRequest> validator, TimeProvider time)
    {
        _db = db; _currentUser = currentUser; _validator = validator; _time =
```

```
time;
    }

    private DateTime UtcNow => _time.GetUtcNow().UtcDateTime;

    public async Task<OrderDto> CreateAsync(CreateOrderRequest request,
CancellationToken ct)
    {
        await _validator.ValidateAndThrowAsync(request, ct);
        var userId = _currentUser.UserId;

        var match = await _db.Matches.FirstOrDefaultAsync(m => m.Id ==
request.MatchId, ct)
            ?? throw new NotFoundException("Match not found");
        if (match.Status != MatchStatus.Scheduled || match.StartsAt <=
UtcNow)
            throw new BusinessRuleException("Tickets for this match are not
on sale");

        var seatIds = request.SeatIds;
        var seats = await _db.Seats.Include(s => s.Sector)
            .Where(s => seatIds.Contains(s.Id) && s.Sector.StadiumId ==
match.StadiumId)
            .ToListAsync(ct);
        if (seats.Count != seatIds.Count)
            throw new NotFoundException("Some seats were not found at this
stadium");

        var anyTaken = await _db.Tickets.AnyAsync(
            t => t.MatchId == match.Id && !t.IsCancelled &&
seatIds.Contains(t.SeatId), ct);
        if (anyTaken)
            throw new ConflictException("Some seats are already taken");

        var order = new Order
        {
            Id = Guid.NewGuid(),
            UserId = userId,
            CreatedAt = UtcNow,
            Status = OrderStatus.Paid
        };
        foreach (var seat in seats)
        {
            order.Tickets.Add(new Ticket
            {
                Id = Guid.NewGuid(),
```

```
                MatchId = match.Id,
                SeatId = seat.Id,
                Price = Math.Round(match.BasePrice *
seat.Sector.PriceMultiplier, 2)
            });
        }
        order.TotalAmount = order.Tickets.Sum(t => t.Price);

        _db.Orders.Add(order);
        try
        {
            await _db.SaveChangesAsync(ct);
        }
        catch (DbUpdateException)
        {
            // сработал уникальный индекс: другой покупатель успел раньше
            throw new ConflictException("Some seats were just taken by
another buyer");
        }

        return await GetByIdAsync(order.Id, ct);
    }

    public async Task<List<OrderDto>> GetMyAsync(CancellationToken ct)
    {
        var userId = _currentUser.UserId;
        var orders = await OrdersWithDetails()
            .Where(o => o.UserId == userId)
            .OrderByDescending(o => o.CreatedAt)
            .ToListAsync(ct);
        return orders.Select(ToDto).ToList();
    }

    public async Task<OrderDto> GetByIdAsync(Guid id, CancellationToken ct)
    {
        var userId = _currentUser.UserId;
        var order = await OrdersWithDetails()
            .FirstOrDefaultAsync(o => o.Id == id && o.UserId == userId, ct)
            ?? throw new NotFoundException("Order not found");
        return ToDto(order);
    }

    public async Task<OrderDto> CancelAsync(Guid id, CancellationToken ct)
    {
        var userId = _currentUser.UserId;
        var order = await _db.Orders.Include(o => o.Tickets)
```

```
            .FirstOrDefaultAsync(o => o.Id == id && o.UserId == userId, ct)
            ?? throw new NotFoundException("Order not found");

        if (order.Status == OrderStatus.Cancelled)
            throw new ConflictException("Order is already cancelled");

        var matchId = order.Tickets[0].MatchId;
        var startsAt = await _db.Matches.Where(m => m.Id == matchId)
            .Select(m => m.StartsAt).FirstAsync(ct);
        if (startsAt - UtcNow < TimeSpan.FromHours(24))
            throw new BusinessRuleException(
                "An order can be cancelled no later than 24 hours before the
match");

        order.Status = OrderStatus.Cancelled;
        foreach (var ticket in order.Tickets) ticket.IsCancelled = true;
        await _db.SaveChangesAsync(ct);

        return await GetByIdAsync(id, ct);
    }

    private IQueryable<Order> OrdersWithDetails() =>
_db.Orders.AsNoTracking()
        .Include(o => o.Tickets).ThenInclude(t => t.Match).ThenInclude(m =>
m.HomeTeam)
        .Include(o => o.Tickets).ThenInclude(t => t.Match).ThenInclude(m =>
m.AwayTeam)
        .Include(o => o.Tickets).ThenInclude(t => t.Seat).ThenInclude(s =>
s.Sector);

    private static OrderDto ToDto(Order o)
    {
        var match = o.Tickets.First().Match;
        return new OrderDto(o.Id, match.Id,
            $"{match.HomeTeam.Name} - {match.AwayTeam.Name}", match.StartsAt,
            o.CreatedAt, o.Status.ToString(), o.TotalAmount,
            o.Tickets.Select(t => new TicketDto(t.Id, t.SeatId,
t.Seat.Sector.Name,
                t.Seat.Row, t.Seat.Number, t.Price,
t.IsCancelled)).ToList());
    }
}
```

Что стоит уметь объяснить:

- Чужой заказ возвращает 404, а не 403: так мы не раскрываем, что такой заказ существует.
- Проверка "место занято" в коде нужна ради понятного сообщения, но настоящая защита это частичный уникальный индекс. Если два запроса прошли проверку одновременно, второй `SaveChanges` упадет, и мы вернем 409.
- `catch (DbUpdateException)` пока слишком широкий (под него попадет любая ошибка записи). Точнее было бы проверять код ошибки PostgreSQL 23505 (нарушение уникальности) в слое Infrastructure. Это отличный пункт для раздела "Что бы улучшил".
- Заказ и все его билеты сохраняются одним `SaveChanges`, а он выполняется в одной транзакции, поэтому отдельная транзакция не нужна.

Регистрация: в `AddApplication` добавьте

```
services.AddScoped<IOrderService, OrderService>();
services.AddSingleton(TimeProvider.System);
```

Контроллер `Api/Controllers/OrdersController.cs`:

```
using FootballTickets.Application.Orders;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace FootballTickets.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orders;
    public OrdersController(IOrderService orders) => _orders = orders;

    [HttpPost]
    public async Task<ActionResult<OrderDto>> Create(CreateOrderRequest
request, CancellationToken ct)
    {
        var order = await _orders.CreateAsync(request, ct);
        return CreatedAtAction(nameof(GetById), new { id = order.Id },
order);
    }

    [HttpGet("my")]
```

```
    public async Task<ActionResult<List<OrderDto>>> GetMy(CancellationToken
ct)
        => Ok(await _orders.GetMyAsync(ct));

    [HttpGet("{id:guid}")]
    public async Task<ActionResult<OrderDto>> GetById(Guid id,
CancellationToken ct)
        => Ok(await _orders.GetByIdAsync(id, ct));

    [HttpPost("{id:guid}/cancel")]
    public async Task<ActionResult<OrderDto>> Cancel(Guid id,
CancellationToken ct)
        => Ok(await _orders.CancelAsync(id, ct));
}
```

Сценарий ручной проверки (с токеном пользователя из шага 11):

1. `GET /api/matches`, возьмите id первого матча.
2. `GET /api/matches/{id}/seats`, возьмите id двух свободных мест из разных секторов.
3. `POST /api/orders` с телом `{"matchId":"...","seatIds":["...","..."]}`: ответ 201, сумма равна сумме цен с учетом множителей секторов.
4. Повторите тот же запрос: ответ 409 (места заняты).
5. `GET /api/orders/my`: заказ виден.
6. `POST /api/orders/{id}/cancel`: ответ 200, статус Cancelled (матч через 7 дней, значит дедлайн в 24 часа не нарушен).
7. Снова повторите покупку тех же мест: теперь 201. Это доказывает, что частичный индекс освобождает места после отмены.
8. В DBeaver выполните `SELECT "SeatId", "IsCancelled" FROM "Tickets"` и посмотрите, как выглядят отмененные и активные билеты.

Коммит: `feat(orders): add ticket purchase and cancellation`.

Шаг 14. Тесты

Тесты показывают, что вы умеете проверять свой код, и это часто отличает проекты, которые читают, от тех, которые пролистывают. Два уровня: быстрые unit-тесты бизнес-правил и интеграционный тест на настоящем PostgreSQL для гонки за место.

Пакеты:

```
dotnet add tests/FootballTickets.Application.Tests reference
src/FootballTickets.Infrastructure
dotnet add tests/FootballTickets.Application.Tests package Moq
dotnet add tests/FootballTickets.Application.Tests package
Microsoft.EntityFrameworkCore.InMemory
dotnet add tests/FootballTickets.Application.Tests package
Microsoft.Extensions.TimeProvider.Testing
dotnet add tests/FootballTickets.Api.IntegrationTests package
Microsoft.AspNetCore.Mvc.Testing
dotnet add tests/FootballTickets.Api.IntegrationTests package
Testcontainers.PostgreSql
```

Unit-тесты OrderService. Файл

`tests/FootballTickets.Application.Tests/OrderServiceTests.cs`:

```
using FootballTickets.Application.Common;
using FootballTickets.Application.Orders;
using FootballTickets.Domain.Entities;
using FootballTickets.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Time.Testing;
using Moq;

namespace FootballTickets.Application.Tests;

public class OrderServiceTests
{
    private static readonly DateTime Now = new(2026, 11, 1, 12, 0, 0,
DateTimeKind.Utc);

    private record Fixture(Match Match, Seat VipSeat, Seat StandardSeat, Guid
UserId);

    private static AppDbContext CreateDb() => new(
        new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString()).Options);

    private static async Task<Fixture> SeedAsync(AppDbContext db, DateTime
startsAt)
    {
        var home = new Team { Id = Guid.NewGuid(), Name = "Home", City = "A"
};
        var away = new Team { Id = Guid.NewGuid(), Name = "Away", City = "B"
};
        var stadium = new Stadium { Id = Guid.NewGuid(), Name = "Arena", City
= "A" };
```

```
        var vip = new Sector { Id = Guid.NewGuid(), Name = "VIP",
PriceMultiplier = 2.0m, Stadium = stadium };
        var std = new Sector { Id = Guid.NewGuid(), Name = "B",
PriceMultiplier = 1.0m, Stadium = stadium };
        var vipSeat = new Seat { Id = Guid.NewGuid(), Row = 1, Number = 1,
Sector = vip };
        var stdSeat = new Seat { Id = Guid.NewGuid(), Row = 1, Number = 1,
Sector = std };
        var user = new User { Id = Guid.NewGuid(), Email = "t@t.t", FullName
= "T", PasswordHash = "x" };
        var match = new Match
        {
            Id = Guid.NewGuid(), HomeTeam = home, AwayTeam = away, Stadium =
stadium,
            StartsAt = startsAt, BasePrice = 20m
        };
        db.AddRange(home, away, stadium, vip, std, vipSeat, stdSeat, user,
match);
        await db.SaveChangesAsync();
        return new Fixture(match, vipSeat, stdSeat, user.Id);
    }

    private static OrderService CreateSut(AppDbContext db, Guid userId)
    {
        var currentUser = new Mock<ICurrentUser>();
        currentUser.Setup(u => u.UserId).Returns(userId);
        return new OrderService(db, currentUser.Object, new
CreateOrderRequestValidator(),
            new FakeTimeProvider(new DateTimeOffset(Now)));
    }

    [Fact]
    public async Task Create_CalculatesTotalUsingSectorMultipliers()
    {
        await using var db = CreateDb();
        var f = await SeedAsync(db, Now.AddDays(10));
        var sut = CreateSut(db, f.UserId);

        var result = await sut.CreateAsync(
            new CreateOrderRequest(f.Match.Id, new() { f.VipSeat.Id,
f.StandardSeat.Id }), default);

        Assert.Equal(60m, result.TotalAmount); // 20 * 2.0 + 20 * 1.0
        Assert.Equal(2, result.Tickets.Count);
    }
```

```
    [Fact]
    public async Task Create_Throws_WhenMatchAlreadyStarted()
    {
        await using var db = CreateDb();
        var f = await SeedAsync(db, Now.AddHours(-1));
        var sut = CreateSut(db, f.UserId);

        await Assert.ThrowsAsync<BusinessRuleException>(() =>
sut.CreateAsync(
            new CreateOrderRequest(f.Match.Id, new() { f.VipSeat.Id }),
default));
    }

    [Fact]
    public async Task Create_Throws_WhenSeatIsAlreadyTaken()
    {
        await using var db = CreateDb();
        var f = await SeedAsync(db, Now.AddDays(10));
        var sut = CreateSut(db, f.UserId);
        var request = new CreateOrderRequest(f.Match.Id, new() { f.VipSeat.Id
});

        await sut.CreateAsync(request, default);

        await Assert.ThrowsAsync<ConflictException>(() =>
sut.CreateAsync(request, default));
    }

    [Fact]
    public async Task Cancel_Throws_WhenLessThan24HoursBeforeMatch()
    {
        await using var db = CreateDb();
        var f = await SeedAsync(db, Now.AddHours(12));
        // заказ создаем "в прошлом", когда продажа еще была возможна
        var earlier = new OrderService(db, Mock.Of<ICurrentUser>(u =>
u.UserId == f.UserId),
            new CreateOrderRequestValidator(),
            new FakeTimeProvider(new DateTimeOffset(Now.AddDays(-5))));
        var order = await earlier.CreateAsync(
            new CreateOrderRequest(f.Match.Id, new() { f.VipSeat.Id }),
default);

        var sut = CreateSut(db, f.UserId);
```

```
        await Assert.ThrowsAsync<BusinessRuleException>(() =>
sut.CancelAsync(order.Id, default));
    }
}
```

Добавьте сами еще 3 теста: отмена освобождает места (после отмены те же места снова можно купить), нельзя отменить чужой заказ (NotFoundException), нельзя купить больше 6 мест (ValidationException). Запуск: `dotnet test`.

Важно знать ограничение: провайдер InMemory не проверяет уникальные индексы и внешние ключи, поэтому он годится для бизнес-логики, но не для проверки защиты от гонки. Для этого нужен интеграционный тест на настоящей БД.

Интеграционный тест на PostgreSQL в контейнере. Нужен запущенный Docker. В конец `Api/Program.cs` добавьте строку, чтобы тесты видели класс Program:

```
public partial class Program { }
```

Фабрика приложения, `tests/FootballTickets.Api.IntegrationTests/ApiFactory.cs`:

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Testcontainers.PostgreSql;

namespace FootballTickets.Api.IntegrationTests;

public class ApiFactory : WebApplicationFactory<Program>, IAsyncLifetime
{
    private readonly PostgreSqlContainer _db = new
PostgreSqlBuilder().WithImage("postgres:17").Build();

    public async Task InitializeAsync()
    {
        await _db.StartAsync();
        // переменные окружения надежнее, чем UseSetting: Program читает
настройки сразу при старте
        Environment.SetEnvironmentVariable("ConnectionStrings__Default",
_db.GetConnectionString());
        Environment.SetEnvironmentVariable("Jwt__Key", "integration-tests-
secret-key-0123456789abcdef");
    }

    protected override void ConfigureWebHost(IWebHostBuilder builder)
```

```
        => builder.UseEnvironment("Development"); // включает миграции и
сидер при старте

    async Task IAsyncLifetime.DisposeAsync()
    {
        await _db.DisposeAsync();
        await base.DisposeAsync();
    }
}
```

Сам тест, `OrderConcurrencyTests.cs`:

```
using System.Net;
using System.Net.Http.Headers;
using System.Net.Http.Json;

namespace FootballTickets.Api.IntegrationTests;

public class OrderConcurrencyTests : IClassFixture<ApiFactory>
{
    private readonly ApiFactory _factory;
    public OrderConcurrencyTests(ApiFactory factory) => _factory = factory;

    private record AuthDto(string Token);
    private record MatchItem(Guid Id);
    private record MatchesPage(List<MatchItem> Items);
    private record SeatItem(Guid Id, bool IsTaken);
    private record SectorItem(List<SeatItem> Seats);

    [Fact]
    public async Task TwoBuyers_SameSeat_OnlyOneSucceeds()
    {
        var anon = _factory.CreateClient();
        var tokenA = await GetTokenAsync(anon, "/api/auth/login",
            new { email = "user@tickets.local", password = "User12345!" });
        var tokenB = await GetTokenAsync(anon, "/api/auth/register",
            new { email = "second@tickets.local", password = "Second12345!",
fullName = "Second" });

        var matches = await anon.GetFromJsonAsync<MatchesPage>
("/api/matches");
        var matchId = matches!.Items[0].Id;
        var sectors = await anon.GetFromJsonAsync<List<SectorItem>>
($"/api/matches/{matchId}/seats");
        var seatId = sectors![0].Seats.First(s => !s.IsTaken).Id;
```

```
        var results = await Task.WhenAll(
            BuyAsync(tokenA, matchId, seatId), BuyAsync(tokenB, matchId,
seatId));

        Assert.Equal(1, results.Count(r => r == HttpStatusCode.Created));
        Assert.Equal(1, results.Count(r => r == HttpStatusCode.Conflict));
    }

    private static async Task<string> GetTokenAsync(HttpClient client, string
url, object body)
    {
        var response = await client.PostAsJsonAsync(url, body);
        response.EnsureSuccessStatusCode();
        return (await response.Content.ReadFromJsonAsync<AuthDto>())!.Token;
    }

    private async Task<HttpStatusCode> BuyAsync(string token, Guid matchId,
Guid seatId)
    {
        var client = _factory.CreateClient();
        client.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Bearer", token);
        var response = await client.PostAsJsonAsync("/api/orders",
            new { matchId, seatIds = new[] { seatId } });
        return response.StatusCode;
    }
}
```

Смысл теста: два покупателя одновременно хотят одно место. Независимо от того, кто успеет раньше, ровно один получит 201 и ровно один 409. Этот тест можно показывать на собеседовании как доказательство, что вы думаете о гонках.

Если тест падает с ошибкой про Jwt:Key или строку подключения, проверьте, что `InitializeAsync` отработал до первого `CreateClient()` (так и происходит с `IClassFixture`).

Коммиты: `test(application): add unit tests for order rules` и `test(api): add concurrency integration test`.

Шаг 15. Логирование (Serilog)

```
dotnet add src/FootballTickets.Api package Serilog.AspNetCore
```

В `Program.cs` сразу после создания `builder`:

```
builder.Host.UseSerilog((context, logger) => logger
    .ReadFrom.Configuration(context.Configuration)
    .WriteTo.Console());
```

И после `UseExceptionHandler()`: `app.UseSerilogRequestLogging();`

В `appsettings.json` добавьте:

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Information",
    "Override": {
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  }
}
```

Это убирает шум от фреймворка и оставляет по одной строке на запрос. Добавьте в `OrderService` пару осмысленных логов (покупка создана, заказ отменен) через `ILogger<OrderService>`: логируйте id заказа и пользователя, но никогда пароли и токены.

Коммит: `feat(api): add serilog request logging`.

Шаг 16. Docker: запуск проекта одной командой

Файл `Dockerfile` в корне репозитория (если у вас .NET 8 или 9, замените 10.0 на 8.0 или 9.0):

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/FootballTickets.Domain/*.csproj src/FootballTickets.Domain/
COPY src/FootballTickets.Application/*.csproj
src/FootballTickets.Application/
COPY src/FootballTickets.Infrastructure/*.csproj
src/FootballTickets.Infrastructure/
COPY src/FootballTickets.Api/*.csproj src/FootballTickets.Api/
RUN dotnet restore src/FootballTickets.Api/FootballTickets.Api.csproj
COPY src/ src/
RUN dotnet publish src/FootballTickets.Api/FootballTickets.Api.csproj -c
Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "FootballTickets.Api.dll"]
```

Сначала копируем только `.csproj` и делаем restore: Docker закэширует слой с пакетами, и пересборка после правок кода будет быстрой. Это тоже можно объяснить на собеседовании.

Файл `.dockerignore`:

```
**/bin/
**/obj/
.git/
.vs/
tests/
```

Обновите `docker-compose.yml`, чтобы поднимались и база, и API:

```
services:
  db:
    image: postgres:17
    container_name: tickets-db
    environment:
      POSTGRES_DB: football_tickets
      POSTGRES_USER: tickets
      POSTGRES_PASSWORD: tickets_dev_pass
    ports:
      - "5432:5432"
    volumes:
      - tickets_pgdata:/var/lib/postgresql/data
```

```
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U tickets -d football_tickets"]
      interval: 5s
      timeout: 5s
      retries: 10

  api:
    build: .
    container_name: tickets-api
    depends_on:
      db:
        condition: service_healthy
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ConnectionStrings__Default:
"Host=db;Port=5432;Database=football_tickets;Username=tickets;Password=ticket
s_dev_pass"
      Jwt__Key: "docker-demo-key-change-me-0123456789abcdef"
    ports:
      - "8080:8080"

volumes:
  tickets_pgdata:
```

Что здесь важно:

- Внутри сети compose API обращается к базе по имени сервиса `db`, а не `localhost`.
- Двойное подчеркивание в `ConnectionStrings__Default` это способ передать вложенную настройку через переменную окружения.
- `depends_on` с `service_healthy` не даст API стартовать, пока база не готова принимать подключения.
- Режим Development включен только ради демо: при старте применяются миграции, заливаются тестовые данные и открывается Scalar. В README честно отметьте, что для продакшена миграции запускают отдельным шагом, а секреты берут из хранилища секретов.

Запуск и проверка:

```
docker compose up --build
```

Откройте http://localhost:8080/scalar/v1 и сделайте вход тестовым пользователем. Предупреждение про HTTPS-перенаправление в логах можно игнорировать (в контейнере только HTTP) или убрать `app.UseHttpsRedirection()` для режима контейнера.

Проверьте на чистой машине сценарий, который увидит проверяющий: клонировать репозиторий, выполнить одну команду, открыть адрес. Если это работает без ваших дополнительных объяснений, README написан правильно.

Коммит: `chore: add dockerfile and full docker-compose`.

Шаг 17. CI в GitHub Actions

Файл `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 10.0.x
      - run: dotnet restore
      - run: dotnet build --no-restore -c Release
      - run: dotnet test --no-build -c Release
```

На раннерах GitHub есть Docker, поэтому интеграционные тесты с Testcontainers пройдут там без дополнительных настроек. Если GitHub предложит более новые мажорные версии `actions/checkout` и `actions/setup-dotnet`, обновите их. Версию dotnet укажите свою.

После пуша откройте вкладку Actions и дождитесь зеленой галочки. В README добавьте бейдж (замените логин):

```
![CI](https://github.com/ВАШ_ЛОГИН/football-tickets-
api/actions/workflows/ci.yml/badge.svg)
```

Если сборка красная, это нормально для первого раза: читайте лог, исправляйте, коммитьте. Но в итоге в main должна остаться зеленая.

Коммит: `ci: add github actions workflow`.

Шаг 18. README, который убеждает

README это то, что проверяющий откроет первым и, скорее всего, единственное, что прочитает полностью. Структура:

1. **Название и бейдж CI**, одна строка о том, что делает проект.
2. **Возможности:** 5-7 пунктов (регистрация и JWT, схема мест, покупка билетов с защитой от двойной продажи, отмена по правилам, роли, пагинация и фильтры).
3. **Стек.**
4. **Быстрый старт:** `git clone`, `docker compose up --build`, адрес Scalar, тестовые логины с пометкой "только для демо".
5. **Архитектура:** схема слоев (картинка или Mermaid-диаграмма) и 3-4 предложения о правилах зависимостей.
6. **Схема БД:** ER-диаграмма (можно экспортировать из DBeaver или нарисовать в Enterprise Architect, который вы уже знаете) и пояснение про частичный уникальный индекс.
7. **Ключевые решения:** почему слои, почему без репозиториев и MediatR, как защищены места от гонки, как устроены ошибки (ProblemDetails), как тестируется.
8. **Запуск тестов:** `dotnet test`, оговорка, что для интеграционных нужен Docker.
9. **Что бы улучшил:** честный список (ниже).
10. **Примеры запросов:** 3-4 curl или фрагмента из .http-файла.

Добавьте в репозиторий файл `requests.http` (или коллекцию Postman) с готовыми запросами: так проверяющий потрогает API за минуту.

Пример раздела "Что бы улучшил":

- резервирование мест на 10 минут с автоматическим освобождением (сейчас покупка мгновенная);
- реальная оплата через платежного провайдера и статусы заказа;
- уточнить перехват ошибки уникальности по коду PostgreSQL 23505;
- refresh-токены и хранение хэшей токенов;
- кэширование списка матчей и схемы мест;
- перенос части правил внутрь сущностей (богатая доменная модель);
- rate limiting на вход и регистрацию;
- health checks и метрики;
- отправка билета на почту через очередь.

Коммит: `docs: write readme with architecture and quick start`.

Часть 4. Финал

4.1 Критерии готовности

Проект можно показывать, когда выполнено все:

- [ ] `docker compose up --build` на чистой машине поднимает рабочий API
- [ ] В Scalar видны все эндпоинты из таблицы 1.5
- [ ] Покупка, повторная покупка того же места (409), отмена, повторная покупка после отмены работают
- [ ] Неавторизованный запрос дает 401, обычный пользователь на админском методе дает 403
- [ ] `dotnet test` зеленый локально, CI зеленый на GitHub
- [ ] В репозитории нет паролей и ключей, кроме явно помеченных dev-значений в compose
- [ ] README с бейджем, схемой, быстрым стартом и разделом "Что бы улучшил"
- [ ] В истории не меньше 15-20 осмысленных коммитов
- [ ] Проект закреплен (pin) в профиле GitHub, у репозитория есть описание и темы (topics): `dotnet`, `aspnet-core`, `ef-core`, `postgresql`, `docker`

4.2 Вопросы, которые вам зададут (подготовьте ответы)

1. Почему такая слоистая архитектура и что будет, если Domain сошлется на Infrastructure?
2. Почему не добавили репозитории и MediatR?
3. Как вы защитились от продажи одного места двум людям? Почему одной проверки в коде мало?
4. Что такое частичный (фильтрованный) уникальный индекс и зачем он здесь?
5. Чем отличаются `AsNoTracking`, `Include` и проекция через `Select`, и где вы что использовали?
6. Как устроена JWT-аутентификация, что внутри токена, где хранится ключ?
7. Почему на неверный email и неверный пароль один и тот же ответ?
8. Почему InMemory-провайдер не подходит для проверки гонок и чем вы это заменили?
9. Что делает `TimeProvider` и зачем он в сервисе?
10. Что вы улучшили бы первым, если бы проект пошел в продакшен?

Если на любой из этих вопросов можете ответить своими словами, проект действительно ваш, и это чувствуется. Если нет, вернитесь к соответствующему шагу и разберитесь, прежде чем показывать проект.

4.3 План по времени (ориентир 2-3 недели)

Дни	Что делаем
1-2	Шаги 0-4: репозиторий, решение, сущности
3-4	Шаги 5-9: БД в Docker, DbContext, конфигурации, миграция, сидер
5-6	Шаги 10-11: ошибки, валидация, регистрация и JWT
7-9	Шаги 12-13: матчи и заказы (самая сложная часть)
10-12	Шаг 14: тесты
13-14	Шаги 15-17: логи, Docker, CI
15-16	Шаг 18: README, схемы, requests.http, доработки из списка "допишите сами"

Можно идти быстрее, но не пропускайте шаги 13 и 14: именно они делают проект убедительным.

4.4 Как оформить проект в резюме

Одной-двумя строками, с цифрами и стеком, например: "Football Tickets API: REST API продажи билетов на футбольные матчи. ASP.NET Core, EF Core, PostgreSQL, JWT, Docker, GitHub Actions. Слоистая архитектура, защита от двойной продажи мест через частичный уникальный индекс, unit- и интеграционные тесты (Testcontainers). Ссылка на GitHub."

Пишите только то, что реально сделали и можете объяснить. Когда проект будет готов, переходим ко второму (бот на C# с фоновой работой) и дорабатываем ShooterGame: вместе три проекта покроют бэкенд, фоновые сервисы и десктоп.